CTF Solutions

Course: CMPS426

Group Members:
Yousef Elessawy 1220300
Mohamed Ashraf 4230167
Ahmed Sameh 4230138
Youssef Afify 1220299

1 CTF 1: Manual Wireshark Solution

1.1 Challenge

Extract a flag from suspicious network traffic on port 4444 using Wireshark.

1.2 Solution Steps

1. Open `traffic.pcapng` in Wireshark.
2. Apply filter: `tcp.port == 4444` to isolate traffic.

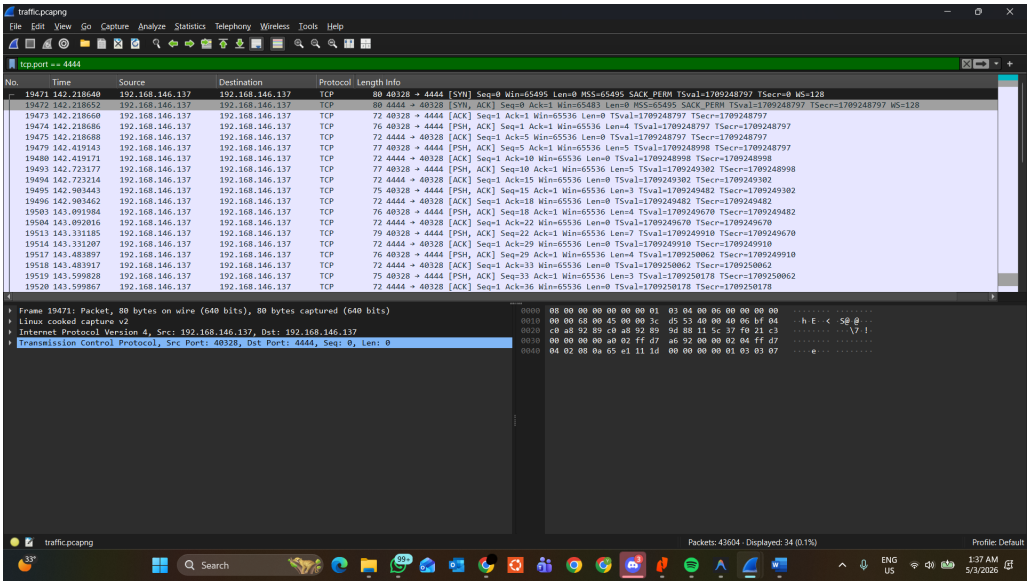

Figure 1: Step 2: Filter the Suspicious Traffic

3. Right-click on a packet and select **Follow → TCP Stream**.
4. Change display format to ASCII.
5. Extract base64 string: `Q01QTntwYzRwX2hpZGQzbl9pbl9sM2dpN183cjRmZmljfQ==`
6. Decode using Python or CyberChef.

1.3 Flag

```
CMPN{pc4p_hidd3n_in_l3gi7_7r4ffic}
```

The flag was wrapped between `MSG:` and `:EOF`, and the middle portion was base64 encoded.

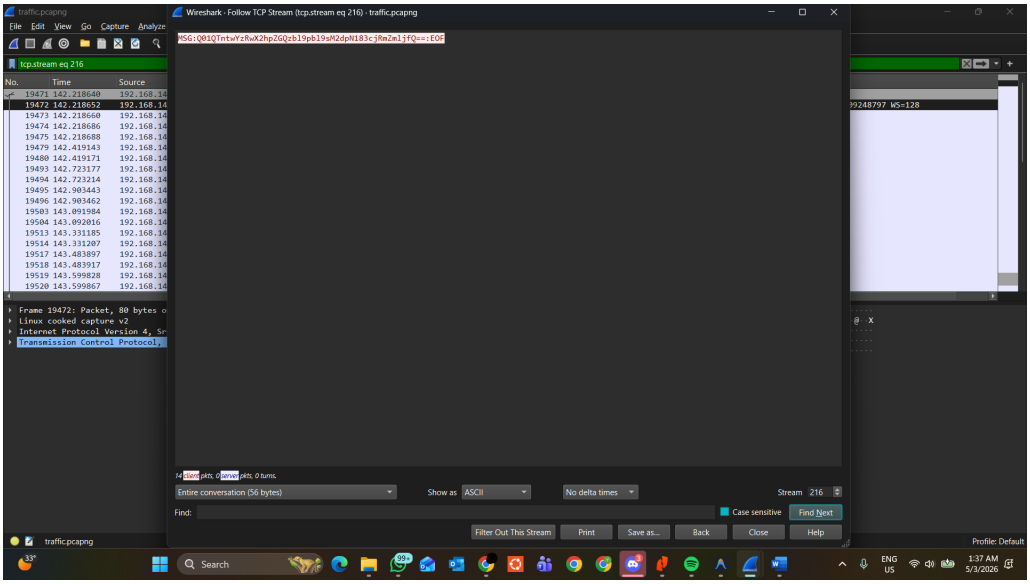

Figure 2: Step 3: Follow the TCP Stream

2 CTF 2: Image Manipulation

2.1 Challenge

Two PNG images (`Layer1.png` and `Layer2.png`) appear as random noise individually. Combine them to reveal a secret.

2.2 Approach

Visual cryptography using XOR: when two noise images are XORed together, randomness cancels out, revealing the hidden content.

2.3 Solution

```
from PIL import Image
import numpy as np

layer1 = Image.open("Layer1.png")
layer2 = Image.open("Layer2.png")

arr1 = np.array(layer1)
arr2 = np.array(layer2)

result = np.bitwise_xor(arr1, arr2)
Image.fromarray(result).save("result.png")
```

2.4 Flag

```
CMPN{im4g3s-4s_k3y$}
```

3 CTF 3: Bit Shifting

3.1 Challenge

Decode a file containing decimal numbers. Each number is a bit-shifted ASCII character.

3.2 Analysis

The file contains:

```
134 154 160 156 246 196 210 110 190 230 208 210 204 110 210 220
   206 190 210 230 190 238 102 104 214 250
```

Since the flag starts with `CMPN{`, the expected ASCII values are:

Character	ASCII	File Value	Relation
C	67	134	$134/2 = 67$
M	77	154	$154/2 = 77$
P	80	160	$160/2 = 80$
N	78	156	$156/2 = 78$
{	123	246	$246/2 = 123$

Each number is exactly twice the ASCII value. The encoding was a **left bit-shift by 1** (`<< 1`). Reverse it with a **right bit-shift by 1** (`>> 1`).

3.3 Decoding Script

```
with open('shifted.txt', 'r') as file:
    shifted_numbers = [int(n) for n in file.read().strip().split
       ()]

flag = "".join(chr(num >> 1) for num in shifted_numbers)
print(flag)
```

3.4 Flag

```
CMPN{bi7_shif7ing_is_w34k}
```

Leetspeak translation: "bit shifting is weak" — a simple bit-shift offers no real security.

4 CTF 4: Steganography

4.1 Challenge

Extract hidden data from `stego.png` using LSB (Least Significant Bit) extraction.

4.2 Approach

The hidden message is encoded in the least significant bit of each grayscale pixel:

1. Load the image using Pillow.
2. Loop through every pixel and extract the LSB: `pixel_value & 1`.
3. Group bits into 8-bit chunks (bytes) and convert to ASCII.
4. Search for the flag format `CMPN{`.

4.3 Solution Pseudocode

```
image = Image.open("stego.png")
pixels = image.load()
bits = []

for y in range(image.height):
    for x in range(image.width):
        bits.append(pixels[x, y] & 1)

text = "".join(chr(int("".join(map(str, bits[i:i+8])), 2))
               for i in range(0, len(bits), 8))
```

4.4 Flag

```
CMPN{Hidd3n_in_pl4in_sigh7}
```

The plaintext message: "Hidden in plain sight." LSB steganography works because the LSBs are the least visible to the human eye.

5 CTF 5: CBC Padding Oracle

5.1 Challenge

Decrypt a ciphertext using a CBC Padding Oracle attack against a server that reveals valid padding.

5.2 Analysis

Target ciphertext (96 hex characters = 3 blocks of 16 bytes):
`b248f0e8f4e3548b995d2215f54b72bd`
`5d3b211b522b7a5ea25c5763e7425447`
`e440e4d85933807e1385d11cd1959975`

The server's `/oracle` endpoint accepts `{"ciphertext_hex": "..."}` and returns `valid_padding`.

5.3 Attack Strategy

In CBC mode: $\text{Plaintext}[N] = \text{Decrypt}(\text{Ciphertext}[N]) \oplus \text{Ciphertext}[N-1]$

By controlling the ciphertext sent to the server, we can:

1. Create a fake previous block and target block.
2. Brute-force the last byte from `0x00` to `0xFF`.
3. When the server returns valid padding, the last byte decrypted to `0x01`.
4. Use XOR to recover the intermediate state: $I = \text{guessed\_byte} \oplus 0x01$.
5. Repeat for all bytes in the block.

5.4 Result

After running the attack:

```
Block 1: CMPN{cbc_p4dding
Block 2: _0r4cl3} + PKCS#7 padding
```

5.5 Flag

```
CMPN{cbc_p4dding_0r4cl3}
```

Tools Used: Python 3, Requests library for HTTP requests

6 CTF 6: RSA Key Recovery

6.1 Challenge

Given RSA public key and ciphertext:

$$n = 143991606075158483660871570161405209117$$
$$e = 65537$$
$$c = 34130411904650996210426832018051041635$$

The primes used to generate n are suspiciously close together.

6.2 Factorization Approach

The modulus is only $\sim$ 128 bits (absurdly small for RSA). When primes p and q are close together, Fermat's factorization or modern techniques (Pollard's p-1, ECM) work efficiently.

Using `sympy.factorint()`:

```
from sympy import factorint

n = 143991606075158483660871570161405209117
factors = factorint(n)
p, q = list(factors.keys())
```

6.3 Factorization Result

$$p = 11607228028223627369$$
$$q = 12405339649142310293$$

The two primes differ by $\sim 6.4\%$, confirming the "too close together" hint.

6.4 Decryption

```
phi = (p - 1) * (q - 1)
d = pow(e, -1, phi)  # modular inverse
m = pow(c, d, n)     # decrypt
flag = m.to_bytes((m.bit_length() + 7) // 8, 'big').decode()
```

6.5 Flag

```
CMPN{f4c70r_m3}
```

Tools Used: Python 3, SymPy for factorization